

SHARDA UNIVERSITY
INVENTION DISCLOSURE FORM
CONFIDENTIAL

IMPORTANT

Please note that patent is always about a solution to an existing technical/industrial problem. As such, draft patent document (IDF) should be crafted in a manner so as to distinctly highlight the existing technical problem, the technical advancement proposed by the inventor(s), and the manner in which the proposed advancement solves the existing technical problem.

➤ **Particulars of Inventors**

S No	Name (Full)	Department / School	Designation	Mobile No.	Email	Official Address
1	Muhammad Shayaan Ali	Department of Computer Science and Engineering	Student	9136182311	2023433557.muhammad@ug.sharda.ac.in	Student, Greater Noida, Sharda University
2	Riddhima Sharma	Department of Computer Science and Engineering	Student	8860850003	2023567320.riddhima@ug.shardac.in	Student, Greater Noida, Sharda University
3	Shruti Tyagi	Department of Computer Science and Engineering	Student	8826043366	2023566902.shruti@ug.sharda.ac.in	Student, Greater Noida, Sharda University
4	Ashish Kumar	Department of Computer Science and Engineering	Assistant Professor	8410707739		

Intelligent DevOps Monitoring & Incident Response System

Abstract : The present invention proposes an Intelligent DevOps Monitoring and Incident Response System for continuously monitoring software infrastructure, identifying abnormal system behaviour, analyzing operational incidents, and assisting in automated incident response. The system collects infrastructure and application monitoring metrics using Prometheus and transmits relevant monitoring events through a real-time communication layer such as a WebSocket-based event channel to an intelligent analysis component.

The intelligent analysis component utilizes a Large Language Model (LLM) to interpret monitoring metrics, alerts, and associated operational information, identify potential incidents, determine their probable causes, and generate an appropriate response or remediation recommendation. The generated response is subsequently passed to a response/incident handling system, which can execute or initiate predefined remediation actions according to the identified incident and configured operational policies.

The proposed system integrates conventional metrics-based monitoring with intelligent event interpretation and response generation, thereby reducing the dependency on continuous manual analysis of alerts. The system is designed to provide a continuous monitoring-to-response pipeline consisting of data collection, real-time event transmission, intelligent incident analysis, response generation, and incident resolution.

The proposed invention aims to improve the efficiency of DevOps operations by reducing alert-analysis time, assisting in identification of probable root causes, and enabling faster and more consistent incident response compared with conventional monitoring systems that primarily generate alerts without intelligent contextual analysis.

Field of Invention: The present invention relates to the fields of DevOps, cloud infrastructure monitoring, software system observability, artificial intelligence, machine learning, and automated incident management.

More particularly, the invention relates to an intelligent monitoring and incident-response mechanism that combines Prometheus-based metric collection, real-time event communication, Large Language Model-based incident analysis, and automated or semi-automated response mechanisms for detecting and handling operational issues in software and computing infrastructure.

Description Or Prior Art :

1) Conventional DevOps Monitoring Systems

Traditional DevOps monitoring systems are mostly used to continuously collect and monitor infrastructure and application metrics such as CPU utilization, memory consumption, network activity, service availability, request rates, response time, and error rates. The monitoring platforms can generate alerts when predefined thresholds or specified monitoring conditions are violated. The ease of monitoring and alert generation is the reason why such systems are widely adopted in DevOps environments. However, conventional monitoring systems mainly provide

observability and alert generation rather than complete intelligent understanding and resolution of incidents. When an alert is generated, DevOps engineers are generally required to manually inspect different metrics, logs, services, and system components in order to determine the underlying cause of the incident. Such manual investigation increases the time required for incident identification and resolution, particularly when multiple system components are involved.

2) Rule-Based Alerting and Incident Management

Existing DevOps monitoring solutions commonly use threshold-based and rule-based alerting mechanisms to identify abnormal system conditions. For example, an alert can be generated when CPU utilization exceeds a predefined threshold, memory usage becomes excessive, service availability falls below a specified level, error rates increase beyond a defined limit, or response latency exceeds an acceptable value. These rule-based mechanisms are effective for identifying known and predefined failure conditions. However, such systems mainly depend on predefined rules and threshold values and may not provide sufficient contextual interpretation of complex incidents involving multiple simultaneously changing metrics. A single alert may also be associated with multiple underlying system conditions, which requires additional manual correlation and investigation by DevOps engineers.

3) Conventional Incident Response

After an alert is generated, incident handling is generally performed through a sequence of manual activities. The DevOps engineer receives the alert and inspects the available monitoring information. The engineer then examines related system behaviour, services, metrics, and logs to identify the probable cause of the incident. After determining the probable cause, an appropriate remediation action is selected and executed. The system is subsequently monitored to determine whether the applied remediation has resolved the incident. This process creates a gap between incident detection and incident resolution. The monitoring system can identify that an abnormal condition has occurred, but the subsequent analysis, decision-making, and response may remain dependent on human intervention.

4) Limitations of Existing Systems

Some of the major limitations of existing DevOps monitoring and incident response approaches are:

- Large numbers of alerts may require manual investigation.
- Correlation of multiple monitoring indicators can be difficult.
- Monitoring alerts may provide limited contextual understanding of an incident.
- Monitoring and incident-response mechanisms are often separated.
- Existing systems may depend heavily on predefined rules for known failure conditions.
- Manual analysis can increase the time between alert generation and incident diagnosis.
- Repetitive incident analysis can increase the workload of DevOps engineers.
- Raw monitoring data may require additional analysis before it can be converted into an actionable response.

- Existing monitoring systems do not inherently provide a continuous integrated pipeline from monitoring data to intelligent incident response.

5) Gap and Motivation Identified

With the above limitations, there is a need for a system capable of providing continuous monitoring, real-time event processing, intelligent incident analysis, and incident response within a unified system. The existing monitoring systems are capable of collecting large amounts of infrastructure and application data. However, the major limitation is the ability to convert such monitoring information into meaningful incident understanding and an appropriate actionable response. Specifically, there is a need for a mechanism that can connect the monitoring and alert generation process with an intelligent analysis system without requiring continuous manual interpretation of monitoring events. The identified gap has motivated the development of the proposed invention, which introduces an intelligent processing layer between the monitoring system and the incident response system.

In the proposed invention, Prometheus is used to collect monitoring metrics from the DevOps infrastructure. Relevant alerts and monitoring events are transmitted through a real-time WebSocket communication layer to an LLM-based analysis component. The LLM-based component analyzes the received monitoring information and available context to identify the incident and its probable cause and generates an appropriate response. The generated response is then provided to a response system, which can execute the corresponding remediation action or involve an authorized human operator where required. The resulting system state is subsequently monitored again to determine whether the incident has been resolved.

Thus, the proposed invention provides an integrated workflow of:

Monitor → Detect → Analyze → Decide → Respond → Verify

This approach combines continuous DevOps monitoring with intelligent incident analysis and response, thereby addressing the gap between **incident detection and incident resolution**.

Disclose working of invention along with drawing, schematics and flow diagrams with explanation

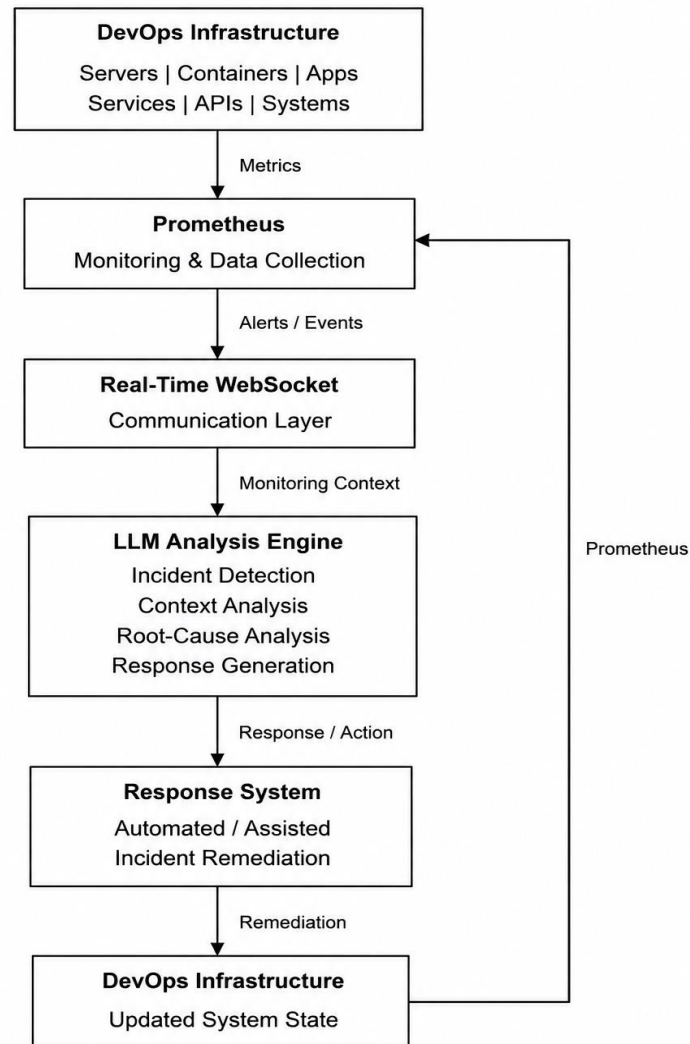


Fig 1. System Architecture

1) Monitoring Data Collection

The process begins with the monitoring of the target infrastructure and software services. Prometheus collects operational metrics from monitored components such as:

- CPU utilization
- Memory utilization
- Disk utilization
- Network-related metrics

- Service availability
- Request rates
- Error rates
- Response latency
- Other configured application/infrastructure metrics

The collected metrics provide the operational state of the monitored environment.

2) Monitoring and Alert Generation

The monitoring data is evaluated according to configured monitoring rules and conditions. When a monitored parameter indicates abnormal behaviour or a configured alert condition is satisfied, an incident-related event is generated.

The event may contain information such as:

- Metric name
- Current metric value
- Threshold/condition
- Timestamp
- Affected service
- Alert status
- Associated monitoring information

3) Real-Time Event Transmission

The generated monitoring information is transmitted through a real-time communication layer, implemented using WebSocket-based communication in the proposed architecture. The communication layer provides a continuous connection between the monitoring infrastructure and the intelligent analysis component. This enables monitoring events to be delivered for analysis without relying exclusively on periodic manual retrieval.

4) Intelligent Incident Analysis

The monitoring event and relevant contextual information are provided to the LLM-based analysis component. The LLM processes the received information and determines the probable nature of the incident.

The analysis can include:

- Identification of abnormal behaviour.
- Interpretation of the affected metric.
- Identification of potentially affected services.
- Correlation of available operational information.
- Estimation of the probable cause.
- Generation of an incident explanation.
- Recommendation of an appropriate response.

5) Incident Classification / Decision

Based on the analysis, the system determines the nature of the detected incident. The generated analysis can be used to categorize the incident according to its observed characteristics and determine whether a predefined response can be applied.

For example:

Monitoring event → Incident identified → Probable cause → Recommended action

6) Response Generation

After analyzing the incident, the intelligent component generates an appropriate response.

The response can include:

- Recommended remediation steps.
- Commands or operational actions.
- Service restart instructions.
- Configuration-related corrective actions.
- Resource-related corrective actions.
- Escalation to a human operator where automated action is not appropriate.

The actual response mechanism should operate according to configured permissions and safety policies.

7) Response System

The generated response is transmitted to a response/incident handling system. The response system is responsible for carrying out the appropriate remediation action or presenting the proposed action to an authorized operator. This separates intelligent decision generation from action execution, allowing the implementation to support automated as well as human-approved responses.

8) Incident Resolution

After the response action is executed, the monitoring system continues observing the infrastructure. The resulting metrics can be used to determine whether the abnormal condition has been reduced or resolved.

Thus, the system creates a feedback relationship:

Response → Monitoring → Verification

9) Continuous Monitoring

After incident handling, monitoring continues without terminating the monitoring process. New abnormal events can again enter the same pipeline:

Prometheus → WebSocket → LLM Analysis → Response System → Verification

This enables continuous operational monitoring and incident handling.

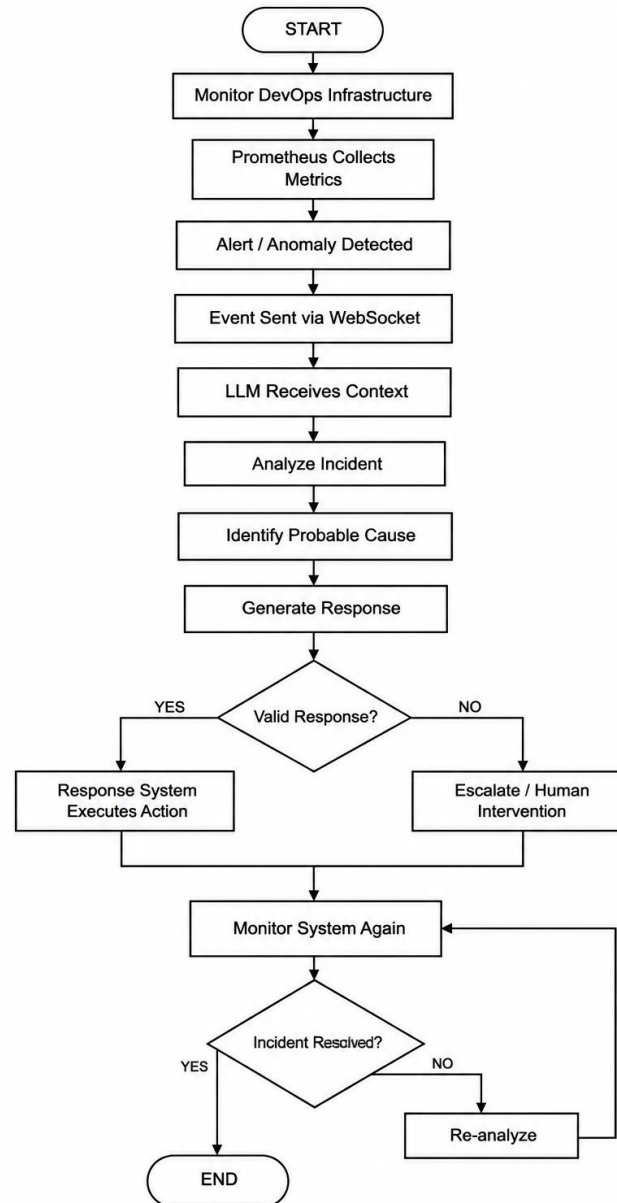


Fig 2. Project Workflow

How it works:

The workflow is initiated when the DevOps infrastructure is continuously monitored by the monitoring system. Prometheus collects the relevant infrastructure and application metrics such as CPU utilization, memory usage, network activity, service availability, request rates, and error

rates. When a predefined monitoring condition or abnormal system behaviour is detected, an alert or event is generated and transmitted through the Real-Time WebSocket Communication Layer for further processing. The monitoring event is then provided to the LLM Analysis Engine along with the available monitoring context. The LLM analyzes the received information to understand the incident, identify the probable cause, and determine an appropriate response. Based on the analysis, the system generates a response or remediation action for the detected incident. The generated response is then passed to the Response System, which can execute the appropriate action automatically or provide the suggested action for human intervention, depending on the configured response mechanism. After the response is applied, the DevOps infrastructure is monitored again by Prometheus to determine whether the system condition has improved and whether the incident has been resolved. If the incident is resolved, the process continues with normal monitoring. If the incident is not resolved, the updated monitoring information is sent back for further analysis and another response can be generated. In this way, the proposed system provides a continuous monitoring, detection, analysis, response, and verification workflow.

GENERAL UTILITY / APPLICATIONS / ADVANTAGES OF THE INVENTION:

1. General Utility

The invention can be used by teams and organizations to continuously monitor their DevOps infrastructure and application systems. It can collect operational metrics, detect abnormal system conditions, and generate alerts for potential incidents. The system can use an LLM-based analysis engine to understand monitoring information, identify the probable cause of an incident, and generate an appropriate response. The system can be useful for organizations that need both continuous infrastructure monitoring and intelligent incident analysis and response. It can reduce the need for manual investigation of repetitive monitoring alerts and provide a continuous mechanism for detecting, analyzing, responding to, and verifying operational incidents.

2. Applications

- Companies and organizations for monitoring internal servers, applications, services, and IT infrastructure.
- Software development and DevOps teams for monitoring development, testing, deployment, and production environments.
- Cloud infrastructure environments for monitoring computing resources, services, and application performance.
- Containerized and microservice-based systems for identifying service failures, performance issues, and abnormal system behaviour.
- CI/CD environments for monitoring application deployment and related infrastructure conditions.
- Enterprise IT environments for monitoring servers, APIs, databases, applications, and network-related services.
- Organizations that require continuous incident monitoring and faster analysis of operational problems.

3) Advantages of Invention

- Continuously monitors infrastructure and application metrics using Prometheus.
- Automatically detects alerts or abnormal system conditions based on configured monitoring conditions.
- Transmits monitoring events through a real-time WebSocket communication layer.
- Provides intelligent analysis of monitoring events using an LLM-based analysis engine.
- Helps identify the probable cause of detected incidents.
- Generates appropriate response or remediation actions based on the analyzed incident.
- Supports automated response as well as human-assisted incident handling.
- The monitoring process does not stop while the incident is being analyzed or responded to.
- The system monitors the infrastructure again after remediation to determine whether the incident has been resolved.
- If the incident is not resolved, the updated monitoring information can be used for further analysis and response.
- Reduces the need for manual investigation and repetitive incident analysis.
- Provides an integrated workflow from monitoring and detection to analysis, response, and verification.
- Reduces the gap between incident detection and incident resolution.

Disclaimer: The undersigned inventor(s) hereby declare that all statements, descriptions, and representations made in this Invention Disclosure Form are true, complete, and made to the best of their knowledge and belief. The inventor(s) shall bear sole responsibility and liability for the accuracy, validity, and authenticity of the information and data provided herein. The Research and Development Cell (RDC), the Institution, and any associated patent attorney, agent, or representative shall not be held liable or accountable for any inaccuracies, omissions, misrepresentations, or consequences arising from the information furnished by the inventor(s).

Signature:

Name: Muhammad Shayaan Ali

Date